COMPUTER NETWORKS PROJECT REPORT

Automated Network Asset Discovery and Service Inventory Framework

A secure, allowlisted network discovery and service inventory system using Python, Nmap, Flask, HTML, CSS, JavaScript, and SQLite.

Project information	Details
Project type	Computer Networks / Cybersecurity application
Primary objective	Discover authorized network assets and maintain a service inventory
Backend	Python with Flask
Scanner	Nmap service/version detection
Database	SQLite
User interface	HTML, CSS, and JavaScript dashboard

Student Name: _____

Register Number: _____

Institution: _____

Academic Year: 2026

Abstract

The Automated Network Asset Discovery and Service Inventory Framework is a web-based Computer Networks project that discovers devices on approved networks, identifies their IP addresses, hostnames, MAC addresses, open ports, and likely services, and stores the results in a structured SQLite database. The framework combines Nmap for network observation, Flask for secure backend APIs, a browser dashboard for monitoring, and a change-detection layer for identifying new devices, unavailable devices, and service changes between scans.

A central safety requirement is that scanning is restricted to explicitly configured authorized CIDR networks. Every requested scan target is parsed and verified as a subnet of the allowlist before Nmap is executed. The system also uses API-key authentication, parameterized SQL, bounded scan execution, explicit error recording, and HTML escaping for displayed scan data.

Keywords: network discovery, asset inventory, Nmap, service detection, Flask, SQLite, change detection, authorized scanning.

1. Introduction

Modern networks contain laptops, servers, routers, printers, virtual machines, and Internet-of-Things devices. Maintaining an accurate inventory is difficult when devices are added, removed, moved, or reconfigured. Manual spreadsheets become outdated quickly and provide little evidence about open services or changes over time.

This project addresses that problem by providing an automated discovery workflow. An operator supplies an approved network range, the scanner collects host and service information, the backend normalizes the result, and the database preserves both current state and historical scan events. A dashboard presents the inventory in a form suitable for operational monitoring and academic demonstration.

2. Problem Statement

Design and implement a controlled framework that can discover active hosts and likely services on authorized networks, preserve the results in a database, expose useful search and monitoring functions, and identify changes between scans without allowing arbitrary or unauthorized network targets.

3. Objectives

• Discover active IPv4 or IPv6 devices within explicitly authorized CIDR ranges.

• Identify IP addresses, hostnames, MAC addresses, vendors, open ports, protocols, and likely service names.

• Store assets, services, scan status, errors, and timestamps in SQLite.

• Detect new devices, devices that no longer respond, added services, removed services, and changed service identification.

• Provide authenticated APIs and a simple dashboard with search, service filtering, status monitoring, and scan controls.

• Provide safe failure behavior when Nmap is missing, times out, returns an error, or produces invalid XML.

4. Scope and Ethical Authorization

The framework is intended for networks owned by the user or networks for which the user has explicit written permission. It is not intended for scanning public networks, third-party systems, or the Internet. The allowlist is not a user-interface suggestion; it is enforced in the backend before the Nmap subprocess starts.

The current scanner uses Nmap service detection with the top 100 ports and a five-minute timeout. This provides a practical project demonstration while limiting scan breadth and runtime. Production deployments should use maintenance windows, a dedicated scanning host, TLS, stronger identity management, auditing, and an approved

security-testing process.

5. System Architecture

Layer	Component	Responsibility
Presentation	HTML/CSS/JavaScript dashboard	Accept API key, start scans, display assets, filter results, show status and changes.
API	Flask routes	Authenticate requests, validate JSON input, return inventory and scan data.
Authorization	CIDR allowlist validator	Ensure requested target is a subnet of an approved network.
Discovery	NmapScanner	Invoke Nmap safely and parse XML output.
Persistence	SQLite database	Store assets, services, scans, and changes.
Automation	Background scheduler	Optionally scan configured networks at a defined interval.

The data flow is: operator or scheduler -> allowlist validation -> Nmap subprocess -> XML parser -> asset/service comparison -> SQLite persistence -> authenticated API -> dashboard.

6. Methodology

6.1 Configuration and validation

At startup, the application loads the .env file. It validates that at least one authorized CIDR exists, the API key is at least 16 characters, and the scan interval is an integer. Relative database paths are resolved under the project directory.

6.2 Authorized target selection

The requested target is parsed using Python's ipaddress module. The request is accepted only when the requested network is a subnet of one of the configured networks. Invalid CIDR values and targets outside the allowlist are rejected with an error response.

6.3 Network discovery

Nmap is executed using an argument list rather than a shell command. The command requests XML output, service/version detection, the top 100 ports, and a controlled timing profile. The subprocess has an explicit timeout and shell=False. Non-zero exit codes, missing executables, timeouts, and invalid XML become recorded scan failures.

6.4 XML normalization

The XML parser keeps hosts marked up, extracts the first IPv4 or IPv6 address, reads the first hostname, reads MAC/vendor information when available, and stores only open ports. Each service includes port, protocol, state, service name, product, and version.

6.5 Persistence and change detection

Before a scan result is applied, currently up assets are temporarily marked down. Hosts found in the new result are restored to up and their last-seen timestamp is updated. A new asset creates a new_device event. Comparing the previous and current service sets creates service_added and service_removed events. Product, version, or service-name differences create service_changed events. Previously up assets absent from the result create device_down events.

7. Inputs

Input	Example	Purpose
API key	MyNetworkScannerKey2026	Authenticates dashboard and API requests.
Authorized network	192.168.1.0/24	Defines the permitted scan boundary.
Scan target	192.168.1.0/24	Network submitted to the scan endpoint; must be allowlisted.
Nmap path	D:/installations/nmap.exe	Location of the Nmap executable on Windows.
Scan interval	0 or 3600	Manual-only mode or automatic scanning interval in seconds.

Example .env configuration

```
ASSET_DISCOVERY_API_KEY=MyNetworkScannerKey2026
ASSET_DISCOVERY_NETWORKS=192.168.1.0/24
ASSET_DISCOVERY_SCAN_INTERVAL_SECONDS=0
ASSET_DISCOVERY_DB_PATH=data/inventory.db
FLASK_SECRET_KEY=replace-with-a-private-secret
NMAP_PATH=D:/installations/nmap.exe
```

The API key shown above is an example for local demonstration. A real deployment must use a unique secret and must not commit .env to source control.

8. Outputs

The dashboard displays an asset table containing IP address, hostname, MAC address, last-seen timestamp, and a compact list of discovered services. It also displays scan status, asset count, recent change count, and a change-history table.

Output	Example
Asset	192.168.1.10, hostname workstation-1, last seen timestamp
Service	22/tcp:ssh, 80/tcp:http
New device event	New device discovered: 192.168.1.10
Service event	Open service added on 192.168.1.10: 8080/tcp
Down event	Device no longer responding: 192.168.1.20
Scan status	running, completed, or failed with error text

9. Database Design

Table	Important columns	Purpose
scans	target, started_at, completed_at, status, error, devices_found	Scan history and operational outcome.
assets	ip_address, hostname, mac_address, vendor, last_seen, status	Current inventory record for each address.
services	asset_id, port, protocol, service_name, product, version, state	Current open service set for an asset.
changes	scan_id, asset_id, change_type, description, created_at	Historical change events.

Foreign keys and indexes are enabled for reliable relationships and efficient service/change queries. Parameterized SQL statements are used for user-controlled values.

10. API and Dashboard Operation

Method and route	Authentication	Function
GET /	None	Serves the dashboard.
GET /api/status	X-API-Key	Returns scan state and latest scan.
GET /api/assets	X-API-Key	Returns inventory; supports q and service filters.
GET /api/changes	X-API-Key	Returns the latest 100 change events.
GET /api/scans	X-API-Key	Returns the latest 50 scan records.
POST /api/scans	X-API-Key	Starts an authorized background scan.

Example scan request

```
POST /api/scans
X-API-Key: <configured-secret>
Content-Type: application/json

{"target": "192.168.1.0/24"}
```

A successful request returns HTTP 202 because Nmap runs in a background thread. The dashboard polls the status endpoint and refreshes inventory and change data every five seconds.

11. Installation and Execution

11.1 Prerequisites

Install Python 3.11 or newer, Nmap for Windows, and Npcap if requested by the Nmap installer. In this implementation, Nmap is installed at D:\installations\nmap.exe.

11.2 Installation commands

```
cd "D:\CN and EH cbp"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

11.3 Start the application

```
cd "D:\CN and EH cbp"
python app.py
```

Open http://127.0.0.1:5000 in a browser. Enter the API key from .env, enter an authorized CIDR, and select Start scan. Keep the terminal window open while the Flask server is running. Stop it with Ctrl+C.

11.4 Verify Nmap

```
& "D:\installations\nmap.exe" --version
```

12. Testing

The project includes unit and integration tests in tests/test_app.py. Nmap is mocked during tests, so test execution does not send network traffic.

Test	Expected result
Nmap XML parsing	Hostnames and service names are extracted correctly.
Unauthorized target	The API returns HTTP 400 and does not start a scan.
Asset persistence	A mocked scan creates an asset record in temporary SQLite.

Test	Expected result
Subprocess safety	Nmap receives an argument list with shell=False.
Full test command	python -m pytest -q; 4 tests pass in the validated project state.

13. Security and Reliability Controls

• CIDR allowlisting prevents arbitrary target scanning.

• API-key authentication protects all inventory and scan endpoints.

• secrets.compare_digest reduces straightforward timing-comparison leakage.

• Nmap runs with shell=False, a bounded timeout, and a controlled argument list.

• SQL parameters are bound rather than concatenated into queries.

• Dashboard values are HTML-escaped before insertion into table markup.

• Errors are returned to the dashboard and recorded in the scans table.

• Database foreign keys and unique constraints preserve inventory consistency.

• The default bind address is 127.0.0.1, reducing accidental network exposure.

14. Advantages

The framework reduces manual inventory effort, creates repeatable evidence of network state, supports quick search, and gives operators a simple view of service exposure. The historical change records help identify newly connected devices and unexpected service changes.

15. Limitations and Future Enhancements

Nmap results depend on network reachability, firewall behavior, privileges, and device response. The current version uses an API key rather than a full user and role system, stores data in a single SQLite database, and scans the top 100 ports for bounded runtime. Future improvements could include role-based authentication, HTTPS deployment, export to CSV/PDF, richer Nmap profiles, IPv6-focused testing, scheduled job persistence, notification integrations, and a dedicated audit log.

16. Conclusion

The Automated Network Asset Discovery and Service Inventory Framework provides a complete and practical solution for authorized network observation. It combines a controlled Nmap scanner, allowlist enforcement, structured persistence, service inventory, change detection, automated scheduling, authenticated APIs, and a responsive dashboard. The implementation demonstrates core Computer Networks concepts including addressing, host discovery, transport ports, service identification, client-server architecture, and network monitoring while maintaining an explicit ethical boundary around authorization.

References

1. Nmap Project, Nmap Network Scanning documentation, https://nmap.org/book/.

2. Pallets Projects, Flask documentation, https://flask.palletsprojects.com/.

3. Python Software Foundation, ipaddress and sqlite3 library documentation, https://docs.python.org/3/.